Glitch Hunter

Panel questions and answers

Sep 29, 2026 · Tanmay Singh

This document lists the questions a panel may ask after the Glitch Hunter presentation, with a clear answer to each. It covers direct and indirect questions, technical and non-technical ones, and questions from people who have never played a video game.

Every number in it comes from the project's own code, test results and records. When an answer admits a weakness, it does so on purpose: an honest limit is easier to defend than a claim that falls apart under a follow-up question.

Key numbers to remember

What	Number
Game	Super Mario Bros, Level 1-1 (a free Python copy of the game)
Buttons the AI can press	10 actions (walk, run, jump, crouch, left, and mixes of these)
What the AI sees	the last 4 screen images, shrunk to 84 × 84 grey pixels
AI brain size	1,689,771 learned numbers (parameters)
Objective 1 brain	6 million practice steps; finishes the level in 46.8% of tries (234 of 500)
Objective 2 brain (final)	16 million practice steps; finishes the level in 56.6% of tries (283 of 500)
Level explored	84.25% of all the places Mario can physically reach (3,166,235 of 3,757,990 pixels)
Bug checks (detectors)	13 rules checked on every game frame
Test bugs planted	6, all found, all replayed, all reported
False alarms on the clean game	0, across 101 AI episodes plus dashboard runs, and 0 in 80 more runs after the stomp check was added
Automated tests in the code	948 tests
Hardware	one laptop with an NVIDIA RTX 3050 Ti GPU; no cloud

1. The project in simple words

Q1. What is Glitch Hunter, in one or two sentences?

It is an AI tester for a video game. An AI learns to play Super Mario, explores the level like a human tester would, and when the game does something it should never do, it stops, saves proof, replays the moment to confirm it, and writes a bug report automatically.

Q2. What problem does it solve?

Testing a game by hand is slow and boring. A person has to play the same level again and again, trying every jump and corner, hoping to see a rare mistake. Glitch Hunter does that playing and watching for you, never gets tired, and writes the report with evidence so a developer can fix the bug quickly.

Q3. Who would use it?

- **Game testers (QA teams):** to cover the boring, repeated play and spend their own time on judgement.
- **Game developers:** to get a bug report with a screenshot, a short video (GIF) and the exact steps to repeat it.
- **Researchers and students:** as a working example of AI-based game testing with a fair way to measure it.

Q4. What are the three objectives?

1. **Objective 1, learn to play:** train an AI that can play and finish Level 1-1 on its own.
2. **Objective 2, learn to explore:** turn that player into a tester that visits as much of the level as possible, without forgetting how to finish it.
3. **Objective 3, find and report bugs:** check the game's rules on every frame, and turn every broken rule into a saved, replayed, fully documented bug report shown on a dashboard.

Q5. What is the full flow, from start to end?

Play → detect → capture → reproduce → report. The AI plays; 13 checks watch every frame; when one fails, the moment is saved (picture, GIF, movement history); the episode is replayed in a separate process to prove the bug comes back; a Markdown and a PDF report are written; the bug appears on the dashboard's Bug Tracker.

Q6. What is the final result, in one line?

A working, end-to-end AI testing system: it found all 6 planted bugs, confirmed each one by replay, reported each one, and raised zero false alarms on the untouched game.

2. Games and bugs, for people who do not play games

Q7. What is Super Mario Bros?

A famous 1985 video game by Nintendo. You control a small character, Mario, who runs from left to right, jumps over gaps and enemies, and tries to reach a flag and a castle at the end of the level. It is one of the best-known games in the world, which makes it easy to explain and easy to check.

Q8. What is "Level 1-1"?

The very first level of the game: World 1, Level 1. It is one long screen that scrolls sideways, with ground, pipes, blocks, stairs, pits and simple enemies, ending at a flagpole and a castle.

Q9. What is a game bug, or a "glitch"?

A moment where the game breaks its own rules. For example: the character walks into a solid wall, is stopped by something that is not there, gets hurt by an enemy that never touched him, or jumps far higher than the game allows. Players notice these and they make a game feel broken or unfair.

Q10. Why do games have so many bugs?

A game is a huge number of moving parts reacting to a player who can do almost anything at any moment. Many bugs only appear for one exact move in one exact place, so they are easy to miss even after hours of testing.

Q11. What does a game tester (QA tester) do?

QA means quality assurance. A tester plays the game on purpose to break it: trying every path, jumping into every corner, repeating strange moves, and writing down anything wrong with steps to repeat it. Glitch Hunter copies this job.

Q12. What are "pipes", "Goombas" and "? blocks"?

Pipes are the green tubes Mario must jump over. A Goomba is the small brown mushroom enemy that walks towards Mario; touching it from the side hurts him. "?" blocks are boxes Mario hits from below to get coins or power-ups. You do not need more than this to follow the project.

Q13. Why use a game for this project and not a normal app?

A game has clear rules of physics (walls are solid, jumps have a maximum height), so a broken rule can be checked by a computer without doubt. It is also a hard, visual, fast-changing environment, which makes it a good test for an AI that has to learn by seeing the screen.

3. Purpose, value and use

Q14. What is the use of this project in the real world?

It shows how game studios could cut the cost and time of testing. An AI tester can play all night, cover places humans skip, and hand the team a ready bug report with proof and replay. Humans then spend their time fixing bugs and judging fun, not repeating the same jump a thousand times.

Q15. Does it replace human testers?

No. It takes over the repetitive part: covering the level and checking hard rules. Humans are still needed for things a rule cannot judge, such as whether a level is fun, fair, or confusing, and for bugs no rule was written for.

Q16. Why not just write a script that plays the level?

A script follows one fixed path, so it only tests that path. It also breaks every time the level changes. The AI learns to play from the screen, reaches places a script writer would not think of, and can be retrained when the level changes instead of being rewritten by hand.

Q17. Why not let the computer press random buttons?

Random play almost never gets far. In Mario, random buttons die at the first Goomba or pit, so the rest of the level is never tested. The AI learned to survive and move forward, which is why it can reach and test the whole level.

Q18. What makes the bug reports useful to a developer?

Each report has: what went wrong, where (exact position), when (exact frame), how serious it is, how sure the system is, the exact trigger picture, a short GIF of the moments before, the full list of button presses from the start of the run, and the result of a replay that shows whether the bug comes back. A developer can repeat it instead of guessing.

Q19. What are the benefits in short?

- Saves tester time on repetitive play.
- Finds rare bugs that need an exact move in an exact place.
- Every bug arrives with proof and a replay, not a vague description.
- Fair measurement: zero false alarms on the clean game is checked, not assumed.

4. The AI: how it learns

Q20. What kind of AI is this?

Reinforcement learning. The AI is not shown correct answers. It tries actions, receives points (rewards) for good results, loses points for bad ones, and slowly learns which actions lead to more points. It is like training a pet with treats.

Q21. Is this ChatGPT-style AI (a language model)?

No. There is no text and no language model. It is a small neural network that looks at game pictures and picks one of 10 buttons. It was trained from zero on this game only.

Q22. What does the AI "see"?

The game screen, turned grey and shrunk to 84 × 84 pixels. It sees the last 4 such pictures together, so it can tell movement and direction (for example, whether Mario is going up or down). It does not read the game's memory to decide what to do.

Q23. What can the AI do? Which buttons does it press?

Ten actions: stand still, walk right, walk right and jump, run right, run right and jump, jump, walk left, crouch, run left, and left and jump. It chooses one action every 4 game frames (the game runs at 60 frames per second, so about 15 choices per game second).

Q24. What is PPO?

Proximal Policy Optimization, a standard and reliable reinforcement learning method. After each batch of play it improves the AI a little, but never too much at once, so learning stays stable. We used the well-tested Stable-Baselines3 version rather than writing our own.

Q25. What is the neural network inside the "brain"?

The standard picture-reading network for this kind of work (a CNN, convolutional neural network): three picture-scanning layers, then one layer of 512 numbers, then two small outputs, one that picks the action and one that estimates how well things are going. In total 1,689,771 learned numbers; the dashboard reads this count from the loaded brain.

Q26. What is a "brain" or "checkpoint" file?

A saved copy of the trained network's numbers in a .zip file. The 6M brain (Objective 1) and the 16M brain (Objective 2, final) are two such files. Loading one lets the dashboard play exactly as that AI learned.

Q27. What does "6M" or "16M" mean?

The number of practice steps the AI took while learning: 6 million and 16 million. One step is one decision (one button choice).

Q28. What is a "reward" and how did you design it?

The points the AI gets. In Objective 1 it got points for moving to new ground, reaching progress marks, and a big bonus for the flag, with a small penalty for dying. In Objective 2 it got points mainly for visiting places in the level it had never visited before, and later in each run for moving towards the finish.

Q29. What training settings did you use?

Final settings: learning rate 0.0001, 2,048 steps per worker per round, batch size 512, 4 passes over each batch, discount 0.99, GAE lambda 0.95, clip range 0.2, entropy (curiosity) weight 0.03, and 8 copies of the game playing in parallel. Each value was set from measured behaviour; the reasons are written next to each one in the code.

Q30. Why 8 games in parallel?

More play per minute. Eight copies of the game run at the same time on the processor, and the network learns from all of them together. This makes training much faster and the learning more stable.

Q31. Does the AI keep learning while it tests on the dashboard?

No. On the dashboard it only plays with the brain it already has. It saves no training files. Learning happens only in the separate training program.

Q32. Does the AI play the same way every time?

No. For each move the brain has a chance for every button combination, and the dashboard draws the move from those chances instead of always taking the top one, so every run takes a new route the brain itself learned. In 100 measured runs on the clean game, 89 reached the castle, each by a different route. The bugged game works the same way, from its first run. The game itself has no randomness.

5. Objective 1: learning to play

Q33. What was done in Objective 1?

We connected the Mario game to a standard AI interface (Gymnasium), defined the 10 actions and what the AI sees, and trained a PPO agent with 8 parallel games for 6 million steps. We also added saving and resuming of training and the first version of the dashboard.

Q34. What was the result?

The 6M brain finishes Level 1-1 in 46.8% of tries (234 of 500 in a fixed 500-run test), with an average progress of 71% of the level. Most failures were deaths to Goombas.

Q35. Why is 46.8% good enough? It fails more than half the time.

The goal was an AI that can play and finish the level on its own, from pixels only, which it does. For a tester, finishing every time is not the main point; reaching and exploring the level is. This brain became the starting point for Objective 2 and the minimum standard: every later brain must finish at least as often as this one.

Q36. What problems did you face in Objective 1?

- The game only allowed a new jump after the jump key was released, but an AI holds keys, so we allow the jump on every frame (documented in the code).
- Parallel game windows opened on top of each other, so window placement was fixed.
- The game loaded its files from the current folder, so paths were made fixed, letting workers start from anywhere.

Q37. How did you measure "finishes the level"?

A fixed test of 500 runs with fixed random seeds, the same for every brain. We count how many reach the flag and castle, the average progress, and how each run ended (level complete, death by Goomba, Koopa or pit, or time-out).

6. Objective 2: exploring like a tester

Q38. Why was a second objective needed if the AI could already play?

A player wants to finish fast; a tester wants to go everywhere. The Objective 1 AI took the shortest path, so large parts of the level (high platforms, corners, gaps) were never tested. Objective 2 taught it to explore while keeping the skill of finishing.

Q39. What is "coverage"?

A map of every pixel of the level where Mario has been. The level is 9,087 pixels long and 600 high. Every time Mario moves, the pixels his body passes through are marked. The map is shared by all 8 training games and kept between runs, so it shows everything the AI has ever tested.

Q40. What does 84.25% coverage mean?

The AI has visited 84.25% of all the places Mario can physically reach: 3,166,235 of 3,757,990 pixels. It is not a percent of the whole picture, because much of the picture (inside walls, high sky) can never be reached.

Q41. How did you know which places Mario can reach?

We built a "reachable map" (the testable mask) using three separate ways of working out reach, plus 228 real jump paths recorded from the game engine, for both small and big Mario. That gives 3,757,990 reachable pixels, which is the fixed number every coverage percent is divided by.

Q42. Why not 100% coverage?

Two reasons. First, 85% of the pixels still missing are high open air that Mario can reach only with rare, perfect jumps. Near solid objects, where bugs actually happen, coverage is 92.14%. Second, gains had become tiny: only +0.27 percentage points in the last 0.8 million steps. We chose 84.25% as a sensible stopping point. We do not claim it is the most possible (our strict upper estimate is 95.31%).

Q43. What went wrong during Objective 2, and how did you fix it?

- **The AI forgot how to finish** around 6.4 million steps: finishing dropped from 43.8% to 6.0%. We went back to the last healthy brain and added "anchor consolidation" (next question).
- **Reward farming:** the AI found it could collect small points by jumping at the same "?" blocks forever. We made those points run out when nothing new is being found.
- **Wrong reachable count (twice):** the map counted places Mario cannot reach. We rebuilt it from real jump paths and moved the saved progress over without losing any.
- **Stuck at tall pipes:** the tallest pipes need a running start. We added a small reward for backing up to take a run-up.

Q44. What is "anchor consolidation"?

A safety rope for the AI's memory. We keep a frozen copy of a healthy brain and a fixed set of game situations. After every learning round we check how differently the new brain would act in those situations. If it has drifted too far (a measured limit of 0.08), we pull it back. This is what stopped the AI from forgetting how to finish the level.

Q45. Why did the AI forget how to finish?

The AI mostly cares about rewards in the near future (roughly the next 100 decisions). The flag is thousands of decisions away, so while chasing new places to explore, the finishing skill slowly faded without any warning in the reward numbers. We found it only because we test every saved brain on the fixed 500-run check.

Q46. What are EXPLORE and COMPLETE?

Two phases inside each run. In EXPLORE the AI is paid for new places. It switches to COMPLETE (paid for moving towards the finish) only when there is evidence that exploring has dried up: its target was met, several time windows found nothing new, or there is nothing reachable left ahead. It never switches just because time passed.

Q47. How do you know the final brain is healthy?

Every saved brain gets the same 500-run finishing test and a label: HEALTHY, WARNING or REGRESSED. The 16M brain finishes 56.6% of runs (283 of 500), better than the 6M brain's 46.8%, so it is HEALTHY. A REGRESSED brain is never used to continue training.

Q48. Why did you stop at 16 million steps?

Coverage was barely growing, finishing had improved and was stable, and the remaining space was mostly high air where bugs are unlikely. More training would cost many hours for very small gains. The brain was frozen with its coverage map and security hashes, and Objective 2 was closed.

Q49. Did you find any real bug in the original game?

Yes, one natural quirk: Mario's speed state from the ground leaks into a jump, so in the air he can go faster than the normal walking speed limit. It can be repeated every time. We did not fix it, because the original game is kept unchanged on purpose; it exists in both game copies and is not part of the 6 test bugs.

7. Objective 3: finding and reporting bugs

Q50. How does the system detect a bug?

After every game frame, 13 checks compare what happened with rules that must always be true. If a rule is broken, that frame is frozen as a "detection". The checks do not know where any bug is; they check the same rule everywhere in the level.

Q51. What are the 13 checks?

- Mario alive below the floor (the fall-death rule failed).
- Mario far above the level.
- Impossible sideways speed.
- Score going backwards; coin count going backwards.
- Mario inside a solid stair, pipe, ground or block (4 checks, more than 6 pixels deep).
- Mario stopped or held up by something that is not drawn (an invisible wall).
- Mario hurt by an enemy that never touched him (not within 2 pixels).
- An enemy stomped although Mario never touched it (not within 2 pixels).
- A jump faster (over 12.5 pixels per frame) or higher (over 258 pixels) than the game's own best real jump.

Q52. How does it know what is "drawn" versus what the game treats as solid?

We stored the level's designed solid objects (ground, pipes, stairs) in a file built from the untouched game, and each frame we also read the live bricks, blocks and enemies that are on screen. The game's own collision boxes are used only as proof that a collision happened; whether it was allowed is judged against what the player can see.

Q53. How did you choose limits like 6 pixels or 258 pixels?

From measurement, not guesses. We recorded normal play and the engine's real fastest and highest jumps, then set limits outside that range so normal play never triggers them. For example, the normal game snaps Mario exactly to a wall's edge, so any overlap deeper than 6 pixels means collision failed.

Q54. What happens at the exact moment a bug is found?

1. The frame is frozen: full-size picture, Mario's recent movement, all button presses since the run began, and the game state.
2. This evidence is written to disk first, before anything else.
3. Testing pauses. On the dashboard the status turns red (*Bug found*), the game screen stops on that frame, and a *Bug found* card appears.
4. In the background, a GIF, a replay check, a Markdown report and a PDF report are made. The five-step list in Live status ticks each stage off as it finishes.
5. The Bug Tracker lists the bug with links to every file.

Q55. What is inside one bug's evidence folder?

Each bug gets its own folder, generated/incidents/INC-<date>-<time>-<id>/, holding: the trigger picture (trigger.png), a GIF of the moments before (context.gif), both reports (report.md, report.pdf), the full record (incident.json), the per-frame movement and the frames (trajectory.json, context_frames.zip), the replay result (reproduction.json), and a list of security fingerprints of every file (manifest.json). The files are never changed afterwards. A bug's folder leaves the disk only when you delete it yourself on the Bug History page.

Q56. How does the replay prove the bug is real?

The game has no randomness and uses its own frame clock, not the real clock. So if you feed it the same button presses from the same start, it does exactly the same thing. The system replays the saved button presses in a separate hidden process and compares every frame: position, speed, score, and whether the same check fires on the same frame. Results are: reproduced, reproduced with state only (pixels differ), not reproduced, diverged, or not possible.

Q57. What if the same bug happens many times?

It is not reported again as a new bug. If the game, the check, the kind of bug and the place are the same, it is counted as another sighting of the same incident, and its "Seen" count goes up.

Q58. How is severity decided?

By the type of bug and its effect on a player. High: being hurt without contact, or falling out of the world while alive (you lose a life or get stuck). Medium: passing into walls, invisible walls, impossible jumps or speed (you can go where the designer did not intend). Low: score or coins going backwards (bookkeeping is wrong but play continues).

Q59. Does the tool also say how to fix a bug?

Yes. Every report and the dashboard's bug card have a "Where to fix it" part: the lines in the game's own code that cause the bug, and the exact edit for each. For example, for the stair bug: "Change line 115 to: step4 = collider.Collider(5874, 366, 40, 176)" (the collision box was 86 pixels shorter than the drawn step); for the brick you can jump through: "Delete lines 554-555" and two more. It works this out from what the incident recorded (the drawn level, the collision boxes in view, the measurements) and by reading the game's source code; it never reads the list of planted bugs. The proof is a test: it applies all eleven suggested edits to a copy of the bugged game, replays every bug's spot, and each one then behaves exactly like the clean game. It is still labelled "inferred, not proven" for new bugs: the developer applies the fix and re-tests.

Q60. What is the "confidence" value in a report?

How strongly the evidence supports the bug. Rules that the engine itself must keep (like "never inside a wall") are exact checks. Limit-based checks (like speed) say how far outside normal play the reading was. Readings in the first two frames after a restart, when the level is still settling, are trusted less.

Q61. What happens when a run on the clean game finishes without bugs?

Testing stops. If Mario reached the castle, a green *Level complete · No bugs found* card appears with a run report (PDF, Markdown, final frame, GIF, or all as a .zip). If he died, the card says so. The run report says plainly that the 13 checks found nothing in that run; it does not claim the whole game is bug-free.

8. The six test bugs, and why the results can be trusted

Q62. What are the six planted bugs?

1. **Stair clipping:** the top blocks of two stair columns are not solid, so Mario sinks into them.
2. **Pipe clipping:** the fourth pipe's solid part starts 32 pixels below its drawn top, so Mario sinks into it.
3. **Ceiling clipping:** one brick does not stop Mario when he jumps up into it.
4. **Invisible wall:** a one-block wall that is not drawn stops Mario on open ground.
5. **Stomp from too far:** one Goomba is stomped from up to 36 pixels above its head, before Mario's feet touch it.
6. **Abnormal sky jump:** a jump from one stretch of ground launches Mario far above the screen.

Q63. Why did you plant bugs yourselves? Isn't that cheating?

It is the standard way to measure a bug finder fairly, used in software testing research. If you do not know where the real bugs are, you cannot tell whether a tool missed any. By planting known bugs, we can measure exactly: did it find all six (yes), and did it raise any alarm on the clean game (no). The checks were written as general rules and do not know where any bug was planted.

Q64. If you already know where the bugs are, why do you need the AI?

We know where they are only to grade the system. The system itself does not know: it must reach each spot by playing, trigger the bug with the right move, and notice the broken rule. In a real game, nobody knows where the bugs are, and the same process would run the same way.

Q65. What is the difference between the clean game and the bugged game?

The clean game is the original, untouched copy; every brain was trained and measured on it. The bugged game is an exact copy with the six bugs added. Every changed line is marked in the code and declared in a list (INJECTED_BUGS.json). The clean game's files are locked by a security fingerprint: any change makes the tests fail, and every report records the fingerprint of the game that produced it.

Q66. Why must the planted bugs never go into the clean game?

The clean game is the reference that proves a report is real. An alarm on the clean game is a false alarm; an alarm on the bugged game at a planted bug is a true detection. If the clean game changed, that comparison would mean nothing, and the AI's measured results would no longer match the game it learned on.

Q67. How do you know there are no false alarms?

We ran the checks on the clean game for 101 AI episodes plus dashboard runs, and got 0 reports. While building the checks, we did see 5 false alarms (for example from bricks that Mario breaks and from the order the game resolves collisions). Each was fixed at its cause and turned into an automated test so it cannot come back.

Q68. Were all six bugs really found?

Yes. On the bugged game, the dashboard stops on all six, each is replayed and marked reproduced, and each gets its GIF, Markdown and PDF report. In a separate test of 40 sampled runs, the checks caught every clip deeper than 6 pixels and never fired where no bug was planted.

Q69. Could the checks find a bug you did not plant?

Yes, if it breaks one of the 13 rules, because the rules are checked everywhere, not only at planted spots. But a bug of a completely different kind (for example, a wrong sound or a wrong colour) would not be caught, because no rule checks for it.

Q70. Why did you place some bugs where they are?

So the agent's own routes would reach them. We measured where its runs go: Mario jumps almost everywhere and walks on the ground only on a few stretches, so the invisible wall stands on one of them (between the first two pits). Our first designs trapped or killed Mario: a two-block wall, a pipe he fell into, and a Goomba that hurt him from a distance ended most runs. We made the wall one block high, let Mario sink into the pipe without being trapped, and turned the Goomba bug into a stomp from too far. Now most runs meet several bugs and still reach the castle.

9. The dashboard: what happens when you click

Q71. What is the dashboard?

A web page, opened in the browser at http://localhost:5000, that runs on your own computer and works without internet. It has four tabs:

- **Overview:** how the agent works, in four steps, and its measured numbers.
- **Live Testing:** the agent playing live, what it is doing, the Bug Found card and the Bug Tracker.
- **How It Works:** four flowcharts (how the agent was built, how it tests each game, what is on the dashboard, your flow from start to end), then what the clean game and the bugged game are and what each is for.
- **Bug History:** every saved bug and every clean-run report, with their files, a Delete button for each and a Clear history button.

It looks like a retro arcade cabinet: black and green, a pixel font, a green-screen log terminal, screen lines like an old monitor on the background (the game screen itself stays clean), and small animations.

Q72. How does the dashboard show what the agent is doing?

- **Live status**, at the right of the game screen. It starts with five steps: agent ready → exploring and checking → bug detected → evidence and replay → report ready. Each step lights up only when it really happens.
- Below the steps, the same panel shows the agent's current action, the steps taken, how far it is to the castle, and the bugs found in this session.
- When a bug is found, the game screen stops on that frame with *BUG FOUND* in its corner, and a *Bug found* card shows each automatic stage as it completes.
- A status label in the top bar (Ready, Testing, Paused, Bug found, No bugs found), always visible, with an icon and a word as well as a colour.

Q73. Are the numbers on the Overview page real, or typed in?

Real. The dashboard reads each one from the project's own files when it starts: the approved Objective-2 record (steps, coverage, completion), the 500-run test results, the declared bug list and the saved evidence. The brain's size on the Live Testing tab is counted from the loaded brain. If something cannot be read, the page leaves it out rather than guess.

Q74. What happens when you start the dashboard?

Double-clicking run_dashboard.bat activates the Python environment and runs app.py. app.py reads the options, sets the laptop to full speed (even on battery), and starts one game thread. That thread picks the brain (the 16M brain if its security hash matches, otherwise the 6M brain), builds the game and loads the brain before the page accepts any click, so the first click is fast. Then the browser opens, centred on the screen.

Q75. After clicking Start testing, which functions and code are called, in order?

1. **Browser (dashboard/web/static/js/main.js):** the button's click handler sends a `start_testing` message over Socket.IO and changes the button to "Testing…".
2. **Server (app.py):** `handle_start_testing()` receives it and calls `service.start_testing()`. It only posts a "start" command; it does not touch the game.
3. **Game thread (dashboard/service.py):** `GameWindowService._run()` takes the command and calls `_handle('start')`, which calls `backend.open_window()` and, if there is no session yet, `backend.new_session()`.
4. **dashboard/backend.py:** `open_agent_window()` makes sure the game and brain exist (`_ensure_global_env_and_model()`) and puts the game window in the taskbar, minimised. `new_session()` starts `run_mario_agent()`, which plays forever, one step at a time.
5. **Every 1/30 of a second** the game thread calls `_step()`, which asks `run_mario_agent()` for one step.

Q76. What happens inside one step?

1. `model.predict()`: the brain looks at the last 4 pictures and picks an action.
2. `env.step(action)` passes through three layers: the frame-skip layer runs the game 4 frames; `GlitchHunterWrapper.step()` (agent_logic.py) records coverage and the reward; `CustomMarioEnv.step()` (custom_mario_env.py) presses the keys, advances the game one frame, reads Mario's position and speed, and runs `_detect_glitches()`, which includes the collision checks in reporting/collision_invariants.py.
3. `encode_frame()` turns the screen into a small JPEG for the browser.
4. `_capture_incidents()` hands any detection to `IncidentPipeline.capture()` (reporting/pipeline.py), which writes the evidence.
5. `step_log_line()` writes the log line, `telemetry()` collects the live-status values, and the step result goes back to the game thread.
6. The game thread sends `video_frame` and `agent_log` to the browser, and `telemetry` every third step; on a bug it pauses and sends `bug_found`; at the end of a clean run it sends `run_finished`.

Q77. How are the reports made without slowing the game?

The raw evidence is written instantly, during the step. The slow parts (GIF, replay, Markdown, PDF) are done by a background worker in reporting/pipeline.py. The replay runs in its own separate process (reporting/reproduce.py). The PDF is drawn with the fpdf2 library (reporting/render.py). When a file is ready, the browser is told (`incident_updated`) and the link changes from a spinner ("PDF report…") to ready.

Q78. How does the live video reach the browser?

Each step, the game screen is shrunk and turned into a JPEG picture, then sent as raw bytes over a WebSocket connection (Socket.IO). The page shows each picture as it arrives. Sending raw bytes instead of text-encoded pictures saves about a third of the data. The live-status values travel the same way, ten times a second.

Q79. Why is there one "game thread"?

On Windows, a window belongs to the thread that created it and dies with it. The web server handles each click on a short-lived thread, so the game window used to vanish or freeze. Now one long-lived thread does every game and window action, and clicks only send it commands. This also guarantees there is only ever one game loop, however fast you click.

Q80. What do Pause, Reset and refreshing the page do?

- **Pause:** stops at once. Nothing is lost; *Resume testing* continues from the same moment.
- **Reset:** ends the session, closes the game window, and clears the log, the Bug Tracker and any result. Saved evidence stays on disk and in Bug History.
- **Refreshing or reopening the page:** does the same as Reset.
- **Closing the game window with its X:** hides it and pauses; *Resume testing* brings it back.

Q81. Can saved reports be deleted?

Yes, on the Bug History page. Every bug and every clean-run report has a Delete button, and each list has a Clear history button. The dashboard asks before it deletes, because the files are removed from disk and cannot be brought back. Three things are kept on purpose: the bug or run that Live Testing is stopped on right now (press Resume or Reset first), a report that is still being written, and anything asked for by another website. If you delete a bug and the AI finds it again, it is saved as a new bug.

Q82. What happens when you switch between the Clean game and the Bugged game?

The dashboard resets, fully unloads the current game (the two copies are never loaded together), loads the other one, and loads the brain against it. Every bug report records which game produced it and that game's fingerprint, so a report can never be blamed on the wrong game.

Q83. Why does the game window open minimised?

You watch the game on the dashboard, so the window would only cover the page. It waits in the taskbar; clicking it there shows it in the centre of the screen. The window is still needed because the game engine draws into it.

Q84. What does run_dashboard.bat change on the laptop?

Only two things, with no admin rights: its own process is not slowed down by Windows' power saving, and the Windows power mode is set to Best performance while it runs. Your own power mode is put back when the dashboard stops.

Q85. How do you stop the dashboard?

Press Esc in its terminal window. The dashboard stops the same way Ctrl+C stops it (the browser disconnects and the power mode is put back), and the window closes by itself. You can also press Esc on the dashboard page and choose *Yes, stop*; the default answer is *No*, so a stray key never stops it. Ctrl+C and closing the window still work. If the dashboard stops because of an error, the window stays open and says what went wrong.

Q86. What if run_dashboard.bat does not start?

When the dashboard stops with an error, run_dashboard.bat runs a check (tools/check_environment.py) that loads every library the dashboard needs and says in plain words what is wrong and how to fix it. setup.bat and setup.sh run the same check before they start the dashboard. One real case: Windows' Smart App Control began blocking PyTorch's files, although nothing in the project had changed. The check names that setting and where to switch it off; reinstalling would not have helped.

Q87. Can the dashboard run on a computer without a GPU or without internet?

Yes to both. The brain only plays (it does not learn) on the dashboard, which a normal processor can do. Everything the page needs is stored locally, so no internet is needed.

Q88. What web requests does the server answer?

The main page, a health check (/healthz), the current status and live values (/api/status), the project's facts (/api/project), the list and details of bugs (/api/incidents), the saved run reports (/api/runs), each bug's files (/incidents/<id>/<file>) and zip bundle, the same for clean-run reports (/runs/…), Bug History's Delete and Clear history (DELETE /api/incidents…, /api/runs…), and the request that stops the dashboard (POST /api/stop, sent when you press Esc on the page and choose Yes). Live events (start, stop, reset, switch game, frames, logs, live status, bug found) go over Socket.IO.

10. Technology choices

Q89. What is the tech stack?

- **Language:** Python 3.12.
- **Game:** Pygame 2.6.1 (the Mario copy is written in it).
- **AI:** Gymnasium 1.3.0 (game-to-AI interface), Stable-Baselines3 2.9.0 (PPO), PyTorch 2.14.0 with CUDA 12.6 (GPU maths).
- **Data and images:** NumPy, OpenCV (pictures and video frames), Pillow (GIFs).
- **Reports:** fpdf2 (PDF) and Markdown.
- **Dashboard:** Flask and Flask-SocketIO on the server; plain HTML, CSS and JavaScript in the browser.
- **Quality:** pytest, Ruff, mypy, pre-commit, GitHub Actions (Windows, Linux and macOS).

Q90. Why Python?

The game is written in Python, and Python has the most mature AI tools (PyTorch, Stable-Baselines3, Gymnasium). One language for the game, the AI, the reports and the server keeps the project simple.

Q91. Why this Mario copy and not the original Nintendo game or an emulator?

It is written in Python, so we can read Mario's exact position and speed, run it frame by frame, control its clock, and create a bugged copy by editing its code. With a game cartridge on an emulator, reading and changing the game this precisely is much harder.

Q92. Why Stable-Baselines3 instead of writing PPO yourselves?

It is a widely used, well-tested library. Writing our own PPO would add risk of hidden mistakes and would not add value to the project's goal, which is testing, not inventing a new learning method.

Q93. Why PPO and not another method like DQN?

PPO is stable, works well with many parallel games, and is a common first choice for games learned from pixels. It also let us use the "keep close to a healthy brain" safety idea (anchor consolidation) cleanly. We did not run a full comparison with other methods; that is a fair piece of future work.

Q94. Why Flask and Socket.IO for the dashboard?

Flask is small and simple for serving a page and files. Socket.IO keeps a live two-way connection, so the server can push video frames, logs, live status and "bug found" events to the browser the instant they happen, without the page asking again and again.

Q95. Why plain JavaScript and not React?

The page is small enough that plain JavaScript needs no build step and no extra downloads, and it keeps the dashboard working fully offline.

Q96. Why is the game shrunk to 84 × 84 grey pictures?

It is the standard size for learning games from pixels. Colour and full size add a lot of computing cost but little useful information for choosing a button. Stacking 4 pictures gives the AI a sense of motion.

Q97. What is Gymnasium?

A standard way to connect any game or simulation to AI code: the AI calls reset() to start and step(action) to act, and gets back a picture, points and whether the run ended. Because our Mario follows it, standard AI libraries work with it directly.

11. Testing, quality and safety

Q98. How did you test your own code?

With 948 automated tests. A single command (tools/check.py) runs style checks (Ruff), strict type checks (mypy), the tests, and checks that the protected brain files have not changed. The same checks run automatically on GitHub for Windows, Linux and macOS on every update.

Q99. How did you test the whole bug-reporting chain end to end?

A validation tool (tools/validate_incident_pipeline.py) runs the real dashboard parts on the clean game with two clearly labelled fake bugs, and checks 37 things: that testing stops, evidence is saved, reports and replay finish, downloads work, resume works, repeats are counted, and the Objective 2 files are untouched. All 37 pass. The dashboard itself was also walked through in a real browser, on both games.

Q100. How do you make sure the brain file was not changed or swapped?

Each protected file has a security fingerprint (SHA-256) recorded in artifacts.json. The dashboard uses the 16M brain only if its fingerprint matches the closure record, and its first progress step says so ("final QA brain"); otherwise it says "Fallback brain" and uses a fallback. If the files are ever missing, the install tool downloads them again and checks all four before writing any of them.

Q101. Is the dashboard safe? Can someone on the network control it?

By default it only listens on your own computer (127.0.0.1), so other devices cannot reach it. Opening it to a network needs an explicit setting and prints a warning, because there is no password. The debug console is always off. Files are served only from a fixed list inside each bug's folder, so tricks like "../" cannot read other files.

Q102. Are there any secrets or passwords in the project?

No. The project has no accounts, tokens or online services. An automatic security scan (Ruff's bandit rules) runs in the checks.

Q103. Are the libraries safe to use?

All versions are fixed exactly. A vulnerability check (pip-audit) on 25 September 2026 found no known problems. We also upgraded PyTorch from 2.5.1 to 2.14.0 because the old version had published security issues; before switching, we checked that both brains made exactly the same moves on both games under both versions.

Q104. What if the program crashes while writing a bug report?

The raw evidence is written first, before testing pauses, and files are written safely (a file is either complete or not there). If rendering a report fails, it is retried and the bug itself is never lost. On restart, unfinished reports are picked up and completed.

12. Results and proof

Q105. How do you prove the system works?

- All 6 planted bugs detected, replayed as reproduced, and reported.
- 0 false alarms on the clean game over 101 episodes plus dashboard runs, and 0 in 80 more runs with the stomp check.
- 37 of 37 end-to-end pipeline checks pass.
- 948 automated tests pass.
- The final brain is measured on a fixed 500-run test and labelled HEALTHY.

Q106. What is the before and after of the AI?

	6M brain (Objective 1)	16M brain (Objective 2)
Finishes the level	46.8% (234 / 500)	56.6% (283 / 500)
Average progress	71%	75%
Greedy (always best move) run	dies after 185 steps	finishes in 434 steps
Reachable level explored	not the goal	84.25%

Q107. How sure are you about 56.6%? Could it be luck?

It is one measurement of 500 runs. Its 95% confidence range is about 52.2% to 60.9%, which is fully above the 6M brain's 46.8%. So the improvement is real, but the exact number could move a few points on another 500 runs.

Q108. How long does it take to find the six bugs?

On the bugged game with the 16M brain, the AI runs into all six within its first episode on the dashboard; testing stops at each one, and you press *Resume testing* to continue to the next.

Q109. How fast is it?

One AI decision takes about 7.6 milliseconds in a training worker (about 130 decisions per second per game). The project's own extra work (coverage, rewards) adds only about 2% on top of the game itself. The dashboard deliberately plays at a steady 30 decisions per second so a human can watch.

13. Limitations and honest weaknesses

Q110. What are the main limitations?

- Only one level (Level 1-1) of one game.
- Coverage is 84.25%, not 100%; places never visited are not proven bug-free.
- The AI still dies in about 25% of runs, so some runs end before the finish.
- Only bugs that break one of the 13 rules are caught; sound, graphics or "fun" problems are not.
- Clips less than 6 pixels deep are ignored on purpose to avoid false alarms.
- The description of what is drawn is built for Level 1-1 only.

Q111. Does "No Bugs Found" mean the game has no bugs?

No. It means that, in that one run, none of the 13 checks fired. It says nothing about places the run did not visit or kinds of bugs no check looks for. The run report states this clearly.

Q112. Does every bugged-game run meet all six bugs?

No, and it does not need to. Every run takes its own route, so it meets its own set of the planted bugs. In 100 measured runs, every one met at least two of the six, 15 met all six, and 74 still reached the castle. The bugs sit where the agent's routes pass, and none of them traps or kills Mario, so a run can meet several and still finish. No run ends stuck. If the agent stops making progress, the dashboard takes over with running jumps and run-ups until Mario reaches new ground, then hands back. The clean game has one pit he could never climb out of (between the first staircase's two columns: 172-pixel walls, and his highest standing jump is 161), so for the last 500 pixels before it the dashboard walks him up the stairs and jumps him across. Out of 100 runs per game, none got stuck. Replays never depend on this: they repeat the recorded button presses.

Q113. What would you do differently if you started again?

- Run the 500-run finishing test on every saved brain from day one, to catch the "forgetting" problem earlier.
- Build the reachable-space map from real jump paths from the start, instead of fixing it twice later.
- Design the game connection (adapter) as a separate, swappable part from the beginning, to make other games easier.

Q114. Did anything fail or get rejected?

Yes. We tried and dropped: re-weighting the rewards, making the AI re-practise finishing (it did worse, 30.2%), a looser memory limit (0.13), a run-up reward alone, and automatically raising exploration pressure when stuck. Each was measured and rejected because it did not help or made things worse.

Q115. Were there any mistakes along the way?

Yes, and they were caught by our own checks: the reachable count was wrong twice, the AI forgot how to finish at 6.4M steps, and the first detectors raised 5 false alarms on the clean game. Each was traced to its cause, fixed, and pinned with a test. Finding and fixing these is part of why we trust the final numbers.

14. Making it generic: other games and other types of games

Q116. Can this be made generic, for any game?

Not fully today, and not for every game. It is built for one level of one game. But it was built in separate layers, and many of them do not depend on Mario at all:

- **Already general:** the learning method (PPO from screen pictures), the coverage idea, the explore-then-finish phases, the evidence and report pipeline, the replay, the duplicate grouping, and the dashboard.
- **Game-specific:** the connection to the game (reading the player's position and speed, mapping actions to keys), the description of what is drawn, the reachable-space map, and the limits of each check (jump height, speed).

Q117. If not for every game, can it be generic for a category of games?

Yes, that is the realistic next step. For 2D side-scrolling platform games (games like Mario, Sonic or Celeste), the same 13 rules still make sense: never pass into a solid, never be stopped by something invisible, never be hurt without contact, never jump beyond the game's own maximum. A new game in this category needs a small adapter that provides the player's box and speed, the list of drawn solid objects and enemies, and its own measured jump and speed limits. The rest can be reused.

Q118. What about 3D games, racing games or puzzle games?

The pipeline, replay, reports and dashboard would carry over, and the AI method still works, but the checks must be new. A racing game needs rules like "the car never passes through a barrier" or "lap time is never impossible"; a puzzle game needs "the score never goes backwards" or "the board never reaches an impossible state". The idea (a clear rule plus exact evidence and replay) is the same.

Q119. Can the trained brain play another game or another level?

No, not directly. It learned from Mario 1-1 pictures only. A different level or game needs new training, though the same training code and settings can be reused, and starting from this brain could speed up learning on similar Mario levels.

Q120. What does a game need for the replay to work?

It must be deterministic: the same button presses from the same start must give exactly the same result. That means the game's clock and any random numbers must be controllable. Our Mario uses its own frame counter and no randomness, which is why every bug can be replayed exactly. Many real games can be made deterministic in a test mode.

15. How someone else can use it for their own game

Q121. How can another person use our tool on the Mario game today?

1. Clone the repository from GitHub (or download it as a ZIP and unzip it).
2. Double-click **setup.bat** (Linux or macOS: `bash setup.sh`). It finds Python 3.12 (on Windows it installs it if missing), installs the fixed library versions, checks the 16M brain that comes with the clone, and opens the dashboard.
3. Open Live Testing, choose the Clean or Bugged game, click *Start testing*, and download the reports.
4. Next time, just double-click run_dashboard.bat.

No GPU, internet (after installing), settings file or training is needed. GitHub runs this same setup on clean Windows, Linux and macOS machines on every update, checks that the dashboard then starts with the approved brain, and on Linux drives the dashboard in a real browser until it finds a bug.

Q122. A person wants to test their own game with it. How?

Today it is a framework that a developer adapts, not a one-click product. The steps are:

1. **Connect the game:** write an adapter like custom_mario_env.py that follows Gymnasium: it presses the game's keys for each action, advances the game one frame, returns the screen, and reports the player's position, speed and state.
2. **Make it repeatable:** let the game run on its own frame clock and fixed random seeds, so replay works.
3. **Describe what is drawn:** produce a file of the level's solid objects (like reporting/level1_design.json) and let the adapter list live enemies and blocks.
4. **Measure normal play:** record the game's real jump height and speed, and set the check limits outside them.
5. **Train a brain:** run train_agent.py on the new game; our PPO settings are a good starting point.
6. **Plug into the dashboard:** register the game as a variant, and the rest (evidence, replay, reports, Bug Tracker) works as it is.

Q123. What if their game is made in Unity or Unreal, not Python?

They would need a bridge between the game and Python. Unity already has one (Unity ML-Agents) that exposes a game to Python AI code in this same style. The game side sends the screen and the player's state; the Python side runs our training, checks and reports. This bridge is not built in our project yet.

Q124. What would make it easy for other people to use?

- Package it as an installable Python library with a clear "game adapter" template.
- Put all per-game values (limits, action list, level description) in one settings file.
- Provide ready adapters for popular engines (Pygame, Unity, Godot).
- Offer ready-made checks for common game types, which users switch on or off.

Q125. Do they have to plant bugs in their own game?

No. Planted bugs were only for us to prove the system works. A user just runs the AI on their real game. Planting a few bugs is still a good way for them to check the setup on their game before trusting it.

16. Similar work, and what is new here

Q126. Does a project like this already exist?

The general idea of AI playtesting exists, mainly as research and inside large studios:

- **EA's SEED research group** published work (2020) on using reinforcement learning agents to find bugs and exploits in game levels, such as places a player can get stuck or pass through.
- **NetEase's "Wuji"** (2019) used evolutionary methods plus deep reinforcement learning to test online combat games and found real bugs.
- **Companies like modl.ai** sell AI bots for game testing, and studios such as King and Ubisoft have described using AI agents in testing.
- **Open tools** such as Unity ML-Agents and gym-super-mario-bros let AI play games, but they are learning tools, not bug finders.

So we do not claim to have invented AI game testing.

Q127. Then what is new or special in your project?

- **The complete loop in one open project:** an AI that explores for coverage, 12 rule-based checks, frozen evidence, automatic replay to confirm, reports and a live dashboard, all connected.
- **Fair measurement:** a clean game and a bugged game with declared, fingerprint-locked bugs, so both "found all bugs" and "no false alarms" are measured, not assumed.
- **Checks against what the player sees:** collisions are judged against what is drawn, not against the game's own (possibly buggy) collision boxes. This is how invisible walls and too-large hit boxes are caught.
- **Exploring without forgetting:** coverage-based exploration with a safety method that kept, and even improved, the ability to finish the level.

Q128. How is this different from gym-super-mario-bros?

gym-super-mario-bros is a popular tool that lets an AI play Mario on an emulator; it is for learning to play. Our project goes further: it explores for coverage, detects bugs, replays them and writes reports. We also use a Python copy of the game, which lets us read exact positions and build a bugged copy.

Q129. How is this different from normal automated testing tools?

Normal tools run steps a person wrote in advance. Our tester decides its own moves by playing, so it can reach situations no one scripted. The rule checks and reports are like normal automated tests, but they run on whatever the AI does.

17. Future scope

Q130. What is the future of this project?

- **More levels and games:** first more Mario levels, then other 2D platform games through a common adapter.
- **Engine plug-ins:** Unity, Godot and Unreal bridges so studios can use it on real games.
- **More kinds of checks:** soft-locks (stuck forever), wrong animations, sound and graphics problems, performance drops.
- **Learned checks:** an AI that learns what "normal" looks like and flags unusual moments, alongside the exact rules.
- **Faster coverage:** several testers with different styles, or curiosity methods that go after hard-to-reach places.
- **Team features:** send reports straight to bug trackers such as Jira or GitHub Issues, with login and multiple users.
- **Nightly testing:** run automatically on every new game build and compare results between builds.

Q131. Could it be used outside games?

The idea can: an AI explores a system, rules check it constantly, and every broken rule is saved, replayed and reported. It could apply to app user interfaces, robot or driving simulators, or any system that can be run step by step and repeated exactly. Each would need its own connection and its own rules.

Q132. Could it become a product or a startup?

The market exists (studios already pay for automated and AI testing). To become a product it would need engine plug-ins, a simple setup for new games, cloud running, and team features. Note that the Mario game code and art cannot be used commercially, so a product would ship without them.

Q133. Can the AI fix the bugs it finds?

Not now. It finds, proves and explains them. Suggesting fixes (for example, "this block has no collision box") is possible future work, because the report already states which rule broke and where.

18. Cost, hardware, time and process

Q134. What hardware did you use?

One laptop with an NVIDIA RTX 3050 Ti laptop GPU, running Windows 11. No cloud computers. The GPU is needed only to train faster; the dashboard runs on any normal computer.

Q135. How long did training take?

Training runs for many hours. The Objective 2 training was done in stages over several days on the laptop, with planned stops for audits, fixes and health checks between stages. Using the finished project needs no training at all.

Q136. How long did the whole project take?

The code history in git runs from 17 August 2026 to late September 2026, about six weeks.

Q137. How much did it cost?

Nothing beyond the laptop and electricity. All software used is free and open source, and no paid cloud or service was used.

Q138. What was the hardest part?

Keeping the AI good at finishing the level while teaching it to explore. It silently forgot how to finish at 6.4 million steps, and fixing that took new measurements (the 500-run test on every brain) and a new method (anchor consolidation). The second hardest was making the bug checks strict enough to catch every planted bug but never fire on the clean game.

Q139. What did you learn?

- In AI training, you must measure the skill you care about directly; reward numbers can look fine while the skill disappears.
- A bug finder is only trustworthy if false alarms are measured on a known-clean version.
- Saving evidence at the exact moment, and proving it by replay, is what makes a bug report useful.

Q140. How did you know when a training change was good?

Every change was measured before and after: coverage growth, reward records per episode, and the 500-run finishing test. Changes that did not improve these were rejected, and each accepted setting is documented in the code with the measurement behind it.

19. Legal and ethical questions

Q141. Is it legal to use Mario?

The game code is a free fan-made copy by Justin Meister, shared for non-commercial, educational use. The pictures, music and sounds belong to Nintendo. Our project is educational and non-commercial, does not claim any link to Nintendo, and says this clearly in its notices. It must not be used commercially with the Mario files.

Q142. What licence does your own code have?

All rights reserved. The code, documents and trained brains are not open source: nobody may use, copy, change or share them without the author's written permission. The source can be read on GitHub, but that gives no right to use it. This notice does not cover the Mario game code, Nintendo's art and audio, or the bundled libraries and font, which keep their own terms.

Q143. Could this be misused, for example for cheating in games?

An AI that finds glitches could in theory help find exploits in online games. Our project only runs on a local, offline copy of a single-player game, and its purpose is to help developers fix bugs before players find them, which reduces cheating rather than enabling it.

Q144. Does it collect any personal data?

No. It runs on your own computer, has no accounts, sends nothing online, and stores only game evidence in local folders.

20. Tricky questions

Q145. Your AI only finishes 56.6% of the time. Is that a good tester?

Yes, because a tester's main job is to reach and test places, not to win. It explores 84.25% of the reachable level and triggered all six planted bugs. Finishing matters so that the last part of the level is tested too, and that ability went up, not down, from 46.8% to 56.6%.

Q146. Isn't a rule-based check too simple to be called AI bug detection?

The AI's job is to play and explore; the checks' job is to judge. Exact rules are chosen for judging on purpose: they are explainable, never guess, and gave zero false alarms. The hard, AI part is getting to every corner and making the right move there, which a fixed script cannot do.

Q147. Did you tune the checks to your own bugs?

No. The checks are general physics rules that apply everywhere. Their limits come from measuring the clean game's normal play, not from the planted bugs. They were also checked on the clean game (0 alarms) and on 40 sampled (slightly random) runs of the bugged game (never fired where there was no bug).

Q148. What if the AI never goes near a bug?

Then that bug is not found. That is why coverage matters and why we report it honestly as 84.25%. Places the AI never visited are not proven bug-free. Better exploration is part of future work.

Q149. Why not just read the game's code to find bugs?

Many bugs come from how parts interact while playing, not from one wrong line you can spot by reading. Also, in real studios the tester often has no access to the code, or the code is huge. Playing and checking behaviour finds what the player will actually experience.

Q150. What if the game changes? Do you need to start over?

The pipeline, checks and dashboard stay the same. The brain may need some extra training for the changed parts, starting from the current brain rather than from zero. The description of what is drawn is rebuilt by a tool (tools/build_level_design.py).

Q151. How do you know the replay is honest and not just repeating the saved result?

The replay does not read the saved outcome to decide. It starts a new, separate game process, feeds it only the saved button presses from the start of the run, and independently watches whether the same check fires on the same frame. It also reports honestly when it cannot confirm: "not reproduced", "diverged" or "not possible" are all real outcomes.

Q152. Is the dashboard showing real data, or is it a prepared demo?

Real data. Every panel reads the live system: the game screen, the agent's action and progress, the five steps, the bug card, the reports and the Overview numbers all come from the running backend and the files on disk. Nothing is hard-coded, and a stage is shown as done only when the backend says it is.

Q153. What if two bugs happen in the same frame?

Each check reports separately, so both are saved as separate incidents with their own evidence, and testing stops once for that step.

Q154. Why does the dashboard pause on every bug? Wouldn't continuing be faster?

Pausing lets a person look at the bug at once, with the game frozen at that moment. The evidence is already saved before the pause, so nothing is lost, and *Resume testing* continues from exactly the same moment.

Q155. Can you show the code that decides a bug?

Yes. The engine checks are in `CustomMarioEnv._detect_glitches()` in custom_mario_env.py, and the collision and jump checks are in reporting/collision_invariants.py (`CollisionInvariants.check()`). Each limit is set in one place, exploration/config.py, with the measurement that justifies it.

Q156. Is the final brain stored in git with the code?

Yes. The final 16M brain and the three files it needs (about 26 MB) are in git, so a fresh copy of the project uses it straight away. Git also stores their security fingerprints, so any copy can be checked. The same files are published as a GitHub Release file as a backup. The older 6M brain is no longer in git; the dashboard needs only the final one.

Q157. Is your result reproducible by someone else?

Yes. Library versions are fixed exactly, the brain files are fingerprint-checked, the game is deterministic, and the tests, the 37-check validation tool and the 500-run finishing test are all included. Anyone can install it and see the same six bugs found on the dashboard.

21. Glossary

Word	Simple meaning
Agent / AI	The program that plays the game by choosing buttons.
Anchor consolidation	A check after each learning round that pulls the AI back if it drifts too far from a known-good version.
Brain / checkpoint	A saved file of the AI's learned numbers.
Bug / glitch	The game breaking its own rules.
Clean game / bugged game	The untouched original, and a copy with six planted bugs.
Coverage	How much of the reachable level the AI has visited.
CNN	A neural network that reads pictures.
Deterministic	Same inputs always give exactly the same result.
Detector / check	A rule tested on every frame, such as "Mario is never inside a wall".
Episode / run	One attempt at the level, from start until finish, death or reset.
False alarm	A bug report when there is no bug.
Frame	One picture of the game; the game shows 60 per second.
Greedy play	The AI always picks its single best move, so every run is the same.
Incident	One recorded bug with all its evidence.
PPO	The learning method used to train the AI.

Word	Simple meaning
QA	Quality assurance: testing a product to find problems.
Reinforcement learning	Learning by trying, getting points for good results and losing points for bad ones.
Replay / reproduction	Running the saved button presses again to prove the bug comes back.
Reward	The points the AI gets while learning.
SHA-256 fingerprint	A code computed from a file; any change to the file changes it.
Socket.IO	A live connection between the server and the browser.
Testable mask	The map of every pixel Mario can physically reach.